Kraków Real Estate Portal - Reasoning

Docplanner recruitment task | Next.js + TypeScript + MySQL + AI-assisted search

> I built the MVP as a vertical slice: acquire real marketplace data, normalize it into MySQL, expose browsing/search/detail pages, then add AI only where it directly improves the user journey: vague-intent search.

Data Acquisition

Source: OLX.pl Kraków apartment sales. I tried Otodom first, but it returned 403, and Sprzedajemy was unreliable from the local environment. OLX worked because each page includes a structured `__PRERENDERED_STATE__` payload.

The scraper collects listing URLs from sale pages, fetches detail pages with polite delays, extracts the embedded JSON, deduplicates by OLX `externalId`, and writes 107 listings to `data/raw/olx-listings.json`.

Fields: title, price, area, rooms, floor, building type, market, furnished flag, district, lat/lng, images, seller info, listing date, and full description. These fields support the core user decisions: affordability, size, location, trust, and visual inspection.

Cleaning & Storage

- **Rooms:** OLX normalized values such as `one`/`two` are mapped to `1`/`2` for filters.
- **Description:** HTML is stripped, but full text is preserved for search and AI interpretation.
- **Missing values:** optional fields stay nullable; I do not drop otherwise useful listings.
- **Deduplication:** scraper-level dedupe plus database `UPSERT` on `externalId`.

Database: MySQL 8 with Prisma 7. The schema indexes the main filter fields: city, district, price, area, and rooms. Prisma's MariaDB adapter needs a runtime `mariadb://` URL while the CLI uses `mysql://`, so the app supports both via environment variables.

Product Surface

The prototype exposes a listing grid with text search, filters, pagination, and a server-rendered detail page. API routes are kept small: `/api/listings` for filtered pagination, `/api/filters` for facets, and `/api/chat-search` for conversational search.

AI Usage

I used AI for intent parsing, not for inventing recommendations. A vague query such as *"nice cheap flat around 40m in Kraków"* is converted into structured filters: price range, area tolerance, room count, district, keywords, and sorting hints. The actual results still come from MySQL.

The prompt includes Kraków-specific context: cheap studio thresholds, common districts, Polish room vocabulary, and ±15% tolerance for approximate area. If `OPENAI_API_KEY` is missing or parsing fails, a regex fallback keeps the same endpoint and UI working.

Keywords are treated as soft signals: they match with OR, and if no listing matches, the system retries without keywords and marks the response as relaxed. This avoids the common AI-search failure mode where subjective words make the query too strict.

Key Assumption

All listings are apartments for sale in Kraków from one OLX crawl in April 2026. Prices are PLN and areas are m². Production multi-source ingestion would require per-source parsers and a shared normalization contract.

Success Metric

Search-to-click rate: percentage of searches where the user opens at least one detail page. Target: more than 70% of searches should produce a clickable result without manual filter correction.

Limitations

1. **Source fragility:** OLX markup or JSON payload could change; field coverage checks should alert on drops.
2. **Small dataset:** 107 listings are enough for MVP behavior, not market completeness.
3. **AI ambiguity:** subjective terms may still be misread, so the UI exposes the parsed interpretation.

With More Time

Scheduled incremental crawling, map view using stored lat/lng, MySQL full-text search, multi-source support, and price-history tracking.